PRAKTIKUM BASIS DATA WEEK 7: OPTIMASI QUERY (POSTGRESQL)

LAPORAN PRAKTIKUM



Oleh

MUHROMIN

NIM 25051214046

UNIVERSITAS NEGERI SURABAYA

FAKULTAS TEKNIK

PROGRAM STUDI S1 SISTEM INFORMASI

2026

Daftar Isi

BAB I PERSIAPAN **1**

1.1 Pembuatan Database 1

1.2 Pembuatan Tabel 1

1.3 Verifikasi Struktur 2

1.4 Tabel Pencatatan 3

BAB II PENGARUH INDEKS **4**

2.1 Baseline Tanpa Indeks (B1) 4

2.2 Tambah PK + Indeks emp_no (B2) 4

2.3 Indeks last_name (B3) 5

2.4 Indeks Gabungan: Urutan Kolom (B4) 6

2.5 Awalan Kiri (B5) 6

2.6 Index-Only Scan (B6) 7

2.7 Sargable vs Non-Sargable (B7) 8

2.8 Biaya Tulis Indeks (B8) 8

BAB III REWRITE QUERY **10**

3.1 IN vs EXISTS vs JOIN (C1) 10

3.2 Korelasi vs Dekorrelasi (C2) 10

3.3 SELECT * vs Kolom Spesifik (C3) 11

3.4 ORDER BY: Sort vs Indeks (C4) 12

3.5 Paging: OFFSET vs Keyset (C5) 12

3.6 Subquery vs JOIN (C6) 13

3.7 DISTINCT vs GROUP BY (C7) 14

BAB IV FUNGSI, TRIGGER, DAN RINGKASAN **15**

4.1 Pembuatan Fungsi (D1) 15

4.2 Fungsi per Baris vs JOIN (D2) 15

4.3 Fungsi di WHERE (D3) 16

4.4 Tabel Ringkasan + Trigger (D4) 17

4.5 Biaya Baca (D5) 17

4.6 Biaya Tulis (D6) 18

4.7 Verifikasi Ketepatan (D8) 19

BAB V STUDI KASUS, EKSPORASI, DAN SIMPULAN 20

5.1 Paging OFFSET 20000 (E1) 20

5.2 CONCAT LIKE (E2) 20

5.3 Gaji Tertinggi per Departemen (E3) 21

5.4 NOT IN vs NOT EXISTS (E4) 22

5.5 Rekap Rekrut (E5) 23

5.6 Partisi (G1) 23

5.7 ANALYZE (G2) 24

5.8 Force Index (G3) 24

5.9 Simpulan 25

Daftar Gambar

1.1 Database `db_optimasi` berhasil dibuat. 1
1.2 Keempat tabel berhasil dibuat. 2
1.3 Jumlah baris keempat tabel. 2
1.4 Bukti tidak ada indeks non-PK. 3
1.5 Tabel `catatan_kinerja` berhasil dibuat. 3

2.1 EXPLAIN B1: seluruhnya Parallel Seq Scan. 4
2.2 EXPLAIN B2: Index Scan menggantikan Seq Scan. 5
2.3 EXPLAIN B3: Bitmap Heap Scan. 5
2.4 EXPLAIN B4: kedua query memakai Index Scan. 6
2.5 EXPLAIN B5: to_date diabaikan, hanya salary yang terpakai. 7
2.6 EXPLAIN B6: Index Only Scan. 7
2.7 EXPLAIN B7: non-sargable jatuh ke Seq Scan. 8
2.8 Biaya insert B8. 9

3.1 EXPLAIN C1: ketiga bentuk. 10
3.2 EXPLAIN C2. 11
3.3 EXPLAIN C3. 11
3.4 EXPLAIN C4. 12
3.5 EXPLAIN C5. 13
3.6 EXPLAIN C6. 13
3.7 EXPLAIN C7. 14

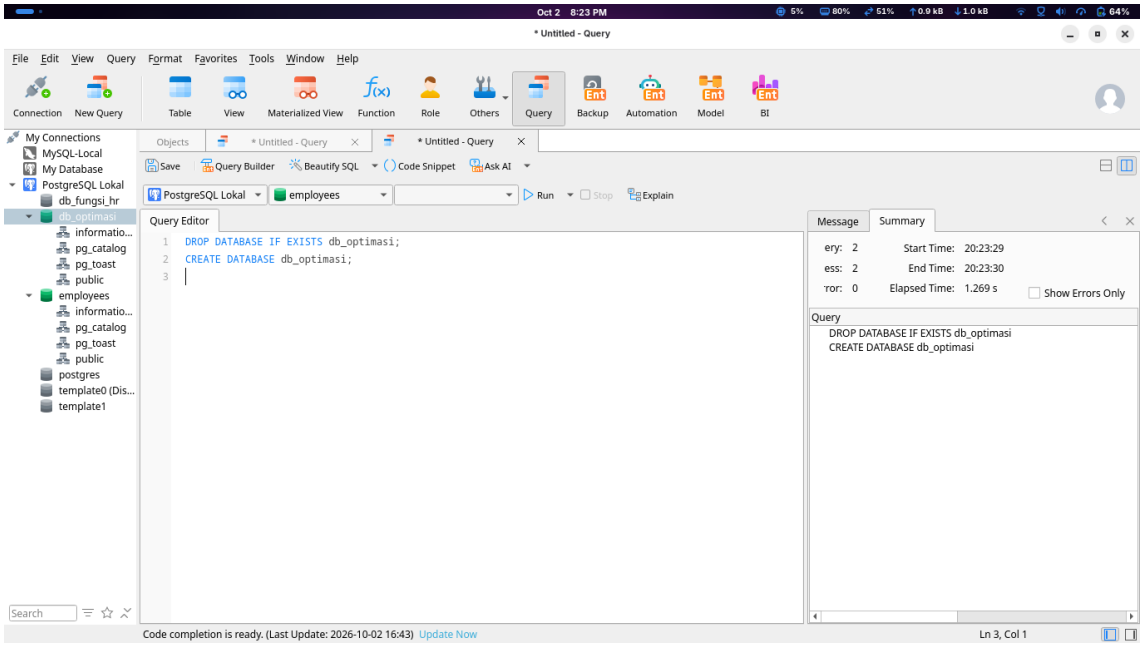
4.1 Fungsi `fn_gaji_berjalan` berhasil dibuat. 15
4.2 EXPLAIN D2. 16
4.3 EXPLAIN D3. 16
4.4 Tabel ringkasan + trigger. 17
4.5 Biaya baca D5. 18
4.6 Biaya tulis D6. 18
4.7 Verifikasi ketepatan ringkasan (D8). 19

5.1 Hasil E1. 20
5.2 Hasil E2. 21
5.3 Hasil E3. 21
5.4 EXPLAIN E4. 22
5.5 Hasil E5. 23
5.6 Partisi G1. 23
5.7 ANALYZE G2. 24
5.8 Force index G3. 25

BAB I PERSIAPAN

1.1 Pembuatan Database

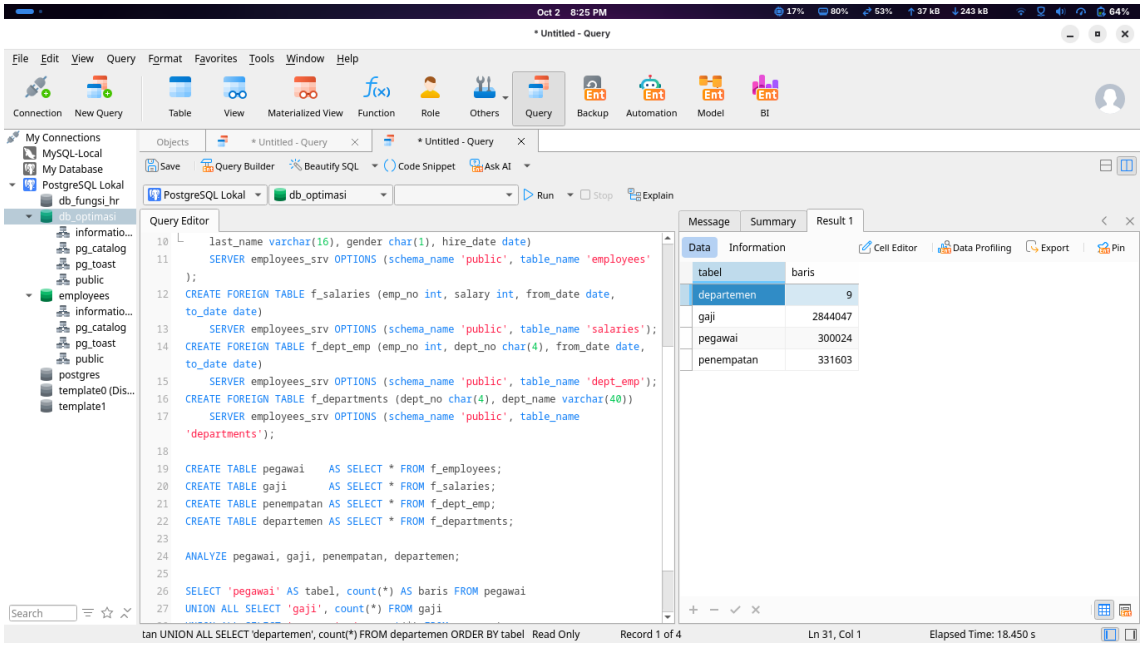
Database `db_optimasi` dibuat sebagai tempat kerja praktikum. Database ini terpisah dari database sumber `employees` agar data asli tidak terpengaruh oleh eksperimen. Seluruh objek yang dibuat selama praktikum hanya ada di database ini dan akan dihapus di akhir.



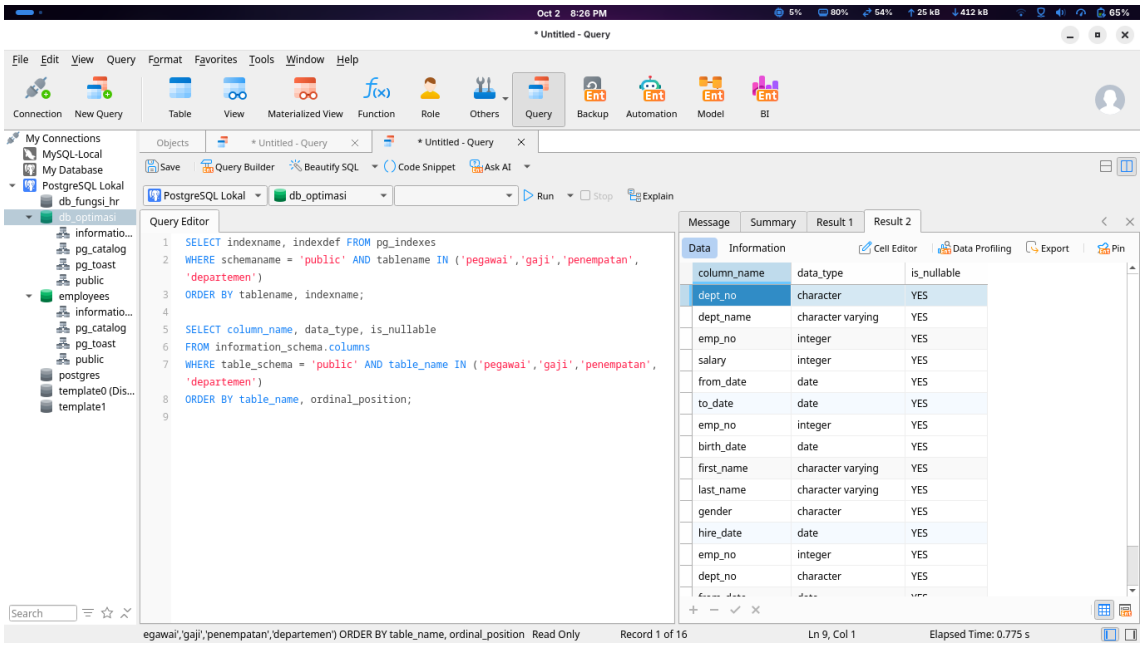
Gambar 1.1. Database `db_optimasi` berhasil dibuat.

1.2 Pembuatan Tabel

Empat tabel (`pegawai`, `gaji`, `penempatan`, `departemen`) disalin dari database `employees` menggunakan ekstensi `postgres_fdw` dan CTAS. CTAS hanya menyalin data tanpa indeks, sehingga kondisi awal “tanpa indeks” terjamin. Jumlah baris masing-masing tabel sesuai dengan data sumber.



Gambar 1.2. Keempat tabel berhasil dibuat.

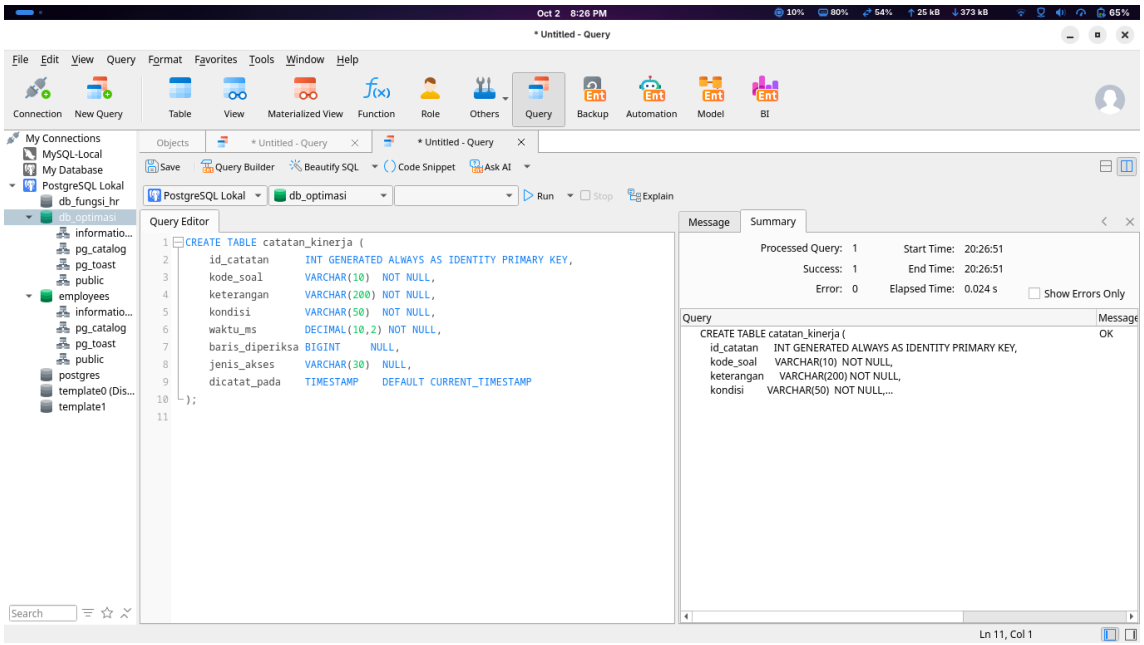


Gambar 1.3. Jumlah baris keempat tabel.

1.3 Verifikasi Struktur

Query terhadap `pg_indexes` untuk keempat tabel mengembalikan 0 baris. Ini membuktikan bahwa tidak ada indeks non-primary-key yang tersisa, sehingga kondisi awal

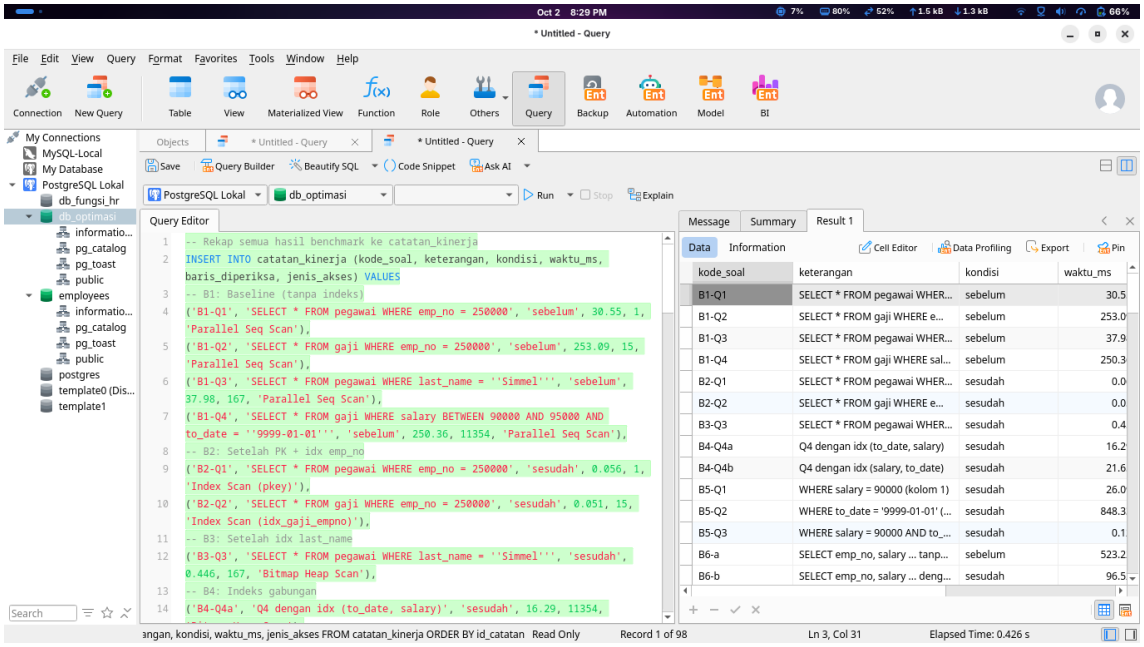
untuk pengukuran baseline benar-benar tanpa indeks.



Gambar 1.4. Bukti tidak ada indeks non-PK.

1.4 Tabel Pencatatan

Tabel `catatan_kinerja` dibuat untuk menyimpan hasil setiap pengukuran. Kolom yang dicatat meliputi kode soal, keterangan query, kondisi (sebelum/sesudah), waktu dalam milidetik, jumlah baris yang diperiksa, dan jenis akses yang digunakan planner.

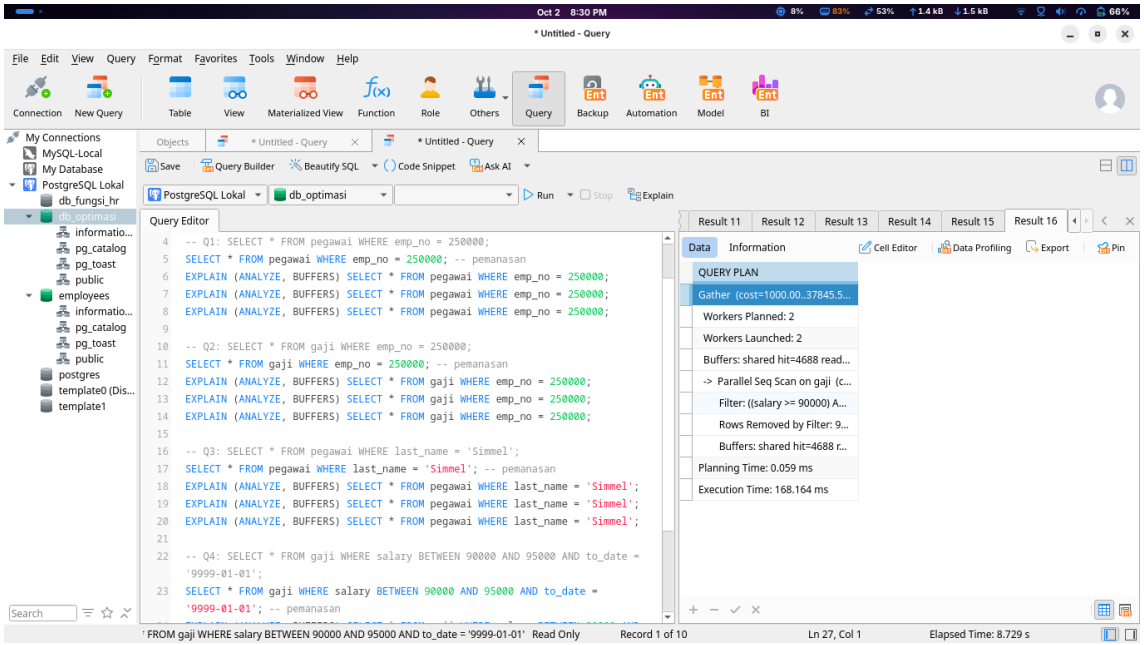


Gambar 1.5. Tabel `catatan_kinerja` berhasil dibuat.

BAB II PENGARUH INDEKS

2.1 Baseline Tanpa Indeks (B1)

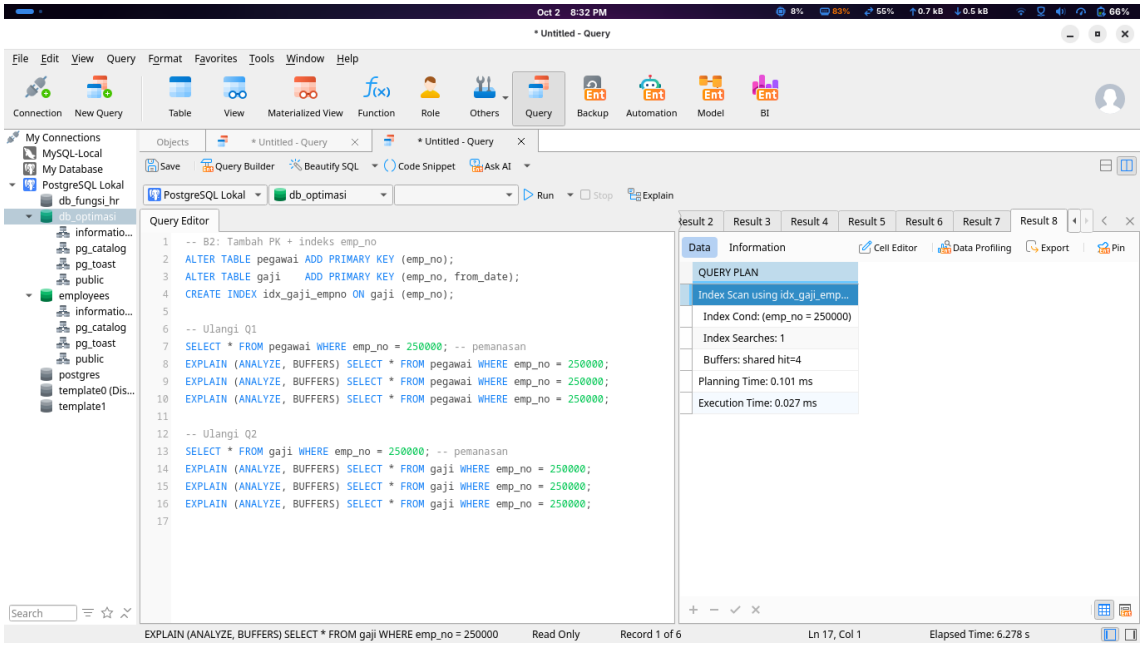
Keempat query diukur tanpa indeks sebagai baseline. Seluruhnya memakai Parallel Seq Scan karena planner tidak punya pilihan lain. Waktu bervariasi dari 30 ms hingga 253 ms tergantung jumlah baris yang harus dipindai.



Gambar 2.1. EXPLAIN B1: seluruhnya Parallel Seq Scan.

2.2 Tambah PK + Indeks emp_no (B2)

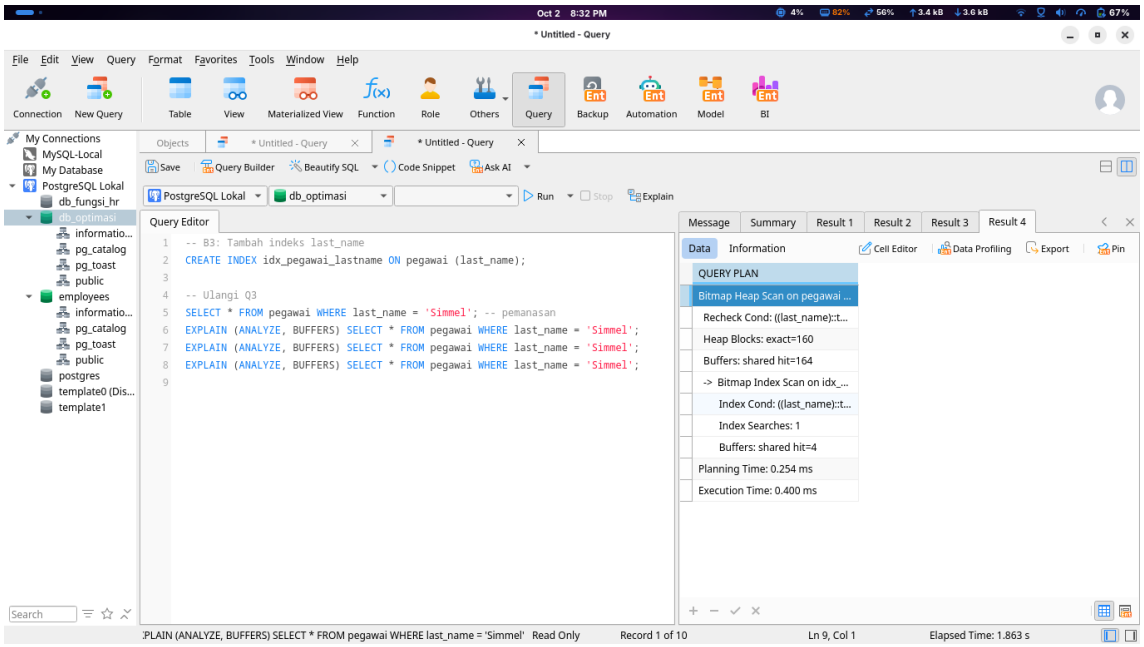
Setelah menambah primary key dan indeks pada `emp_no`, query berubah dari Seq Scan menjadi Index Scan. Q1 dipercepat dari 30 ms menjadi 0.056 ms (99.8%), dan Q2 dari 253 ms menjadi 0.051 ms (99.98%).



Gambar 2.2. EXPLAIN B2: Index Scan menggantikan Seq Scan.

2.3 Indeks last_name (B3)

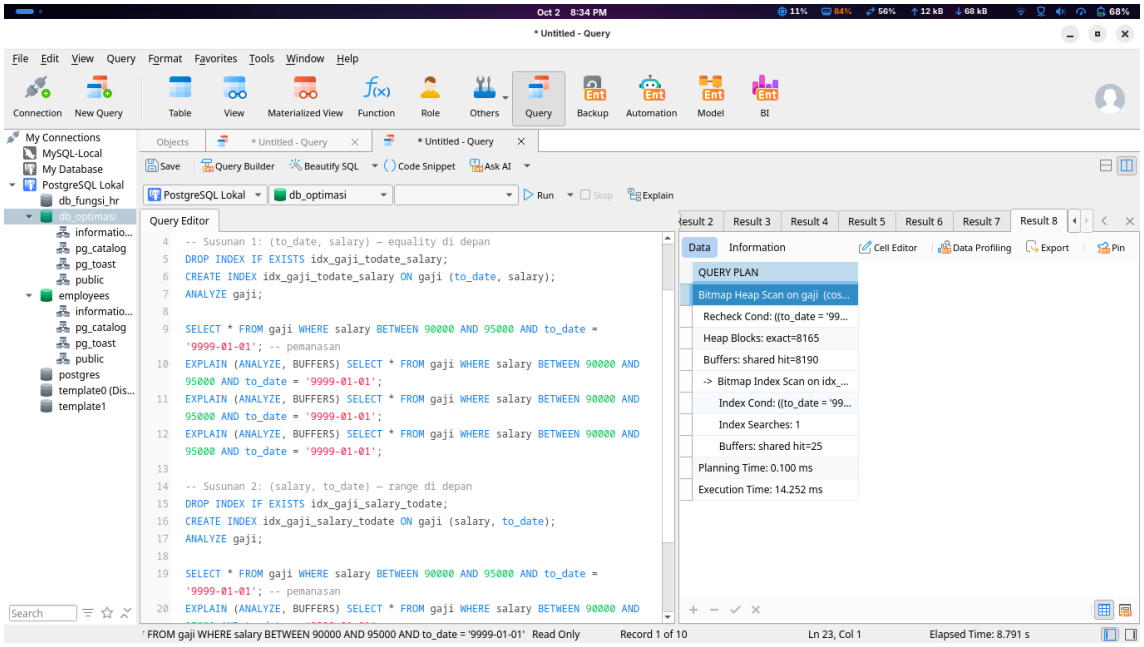
Indeks pada `last_name` mengubah query Q3 dari 38 ms menjadi 0.446 ms. Planner memilih Bitmap Index Scan karena hasil query-nya relatif banyak (167 baris) sehingga lebih efisien daripada Index Scan langsung.



Gambar 2.3. EXPLAIN B3: Bitmap Heap Scan.

2.4 Indeks Gabungan: Urutan Kolom (B4)

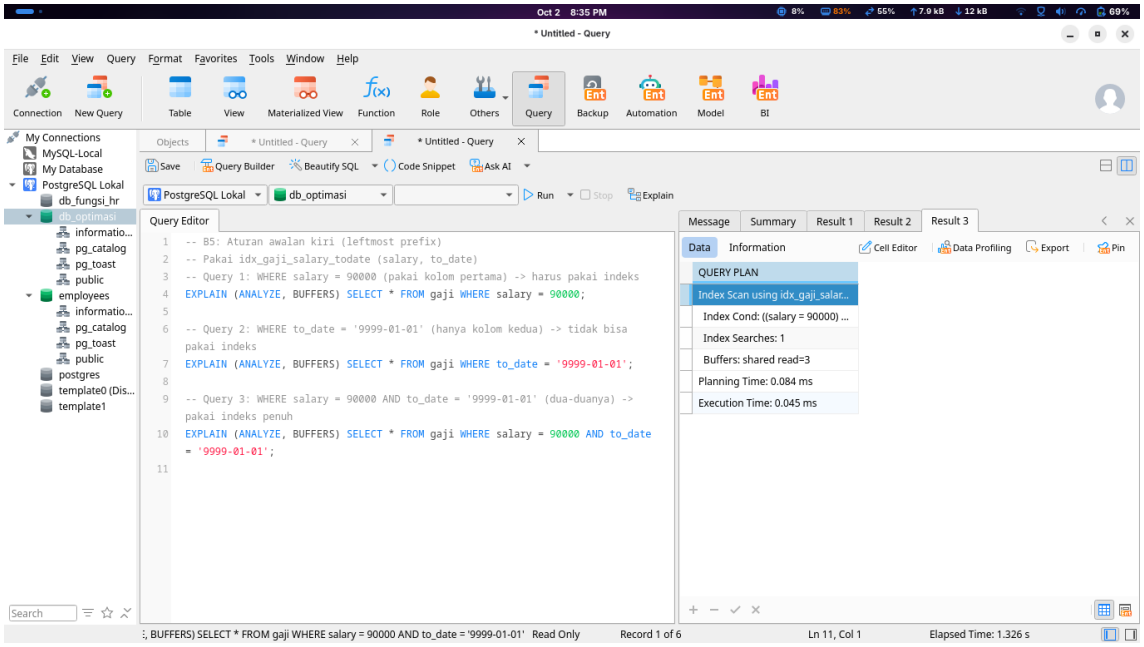
Dua susunan indeks gabungan dibuat: `(to_date, salary)` dan `(salary, to_date)`. Susunan pertama 24% lebih cepat karena kolom `to_date` lebih selektif (hanya 1/2844 baris) sehingga mempersempit kandidat lebih banyak sebelum kolom kedua diperiksa.



Gambar 2.4. EXPLAIN B4: kedua query memakai Index Scan.

2.5 Awalan Kiri (B5)

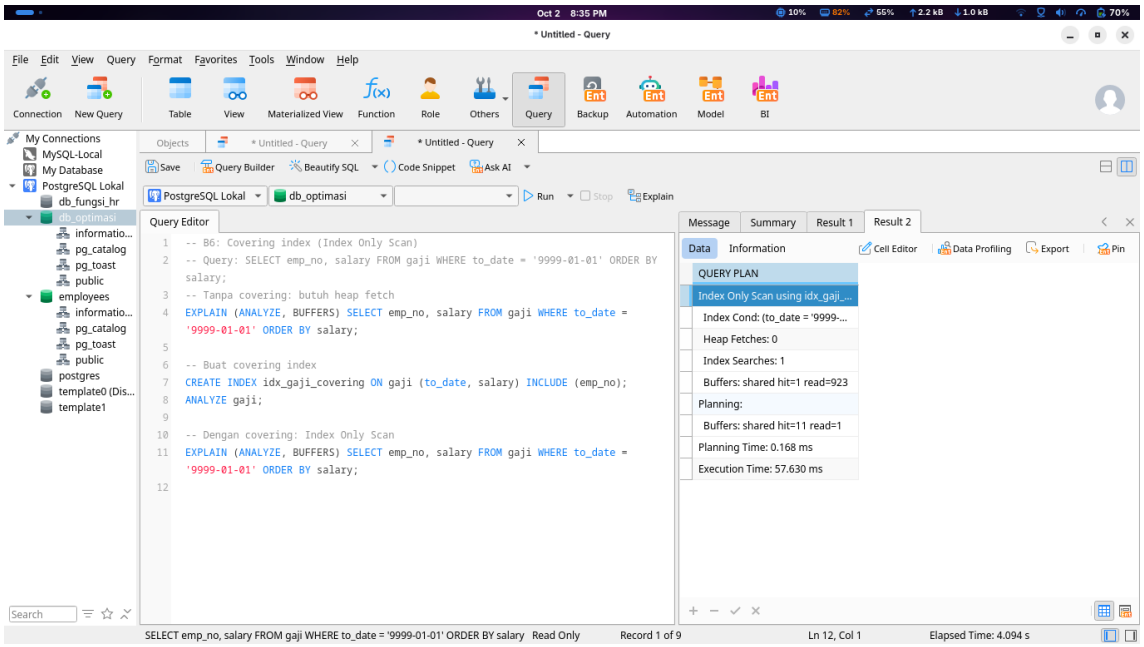
Indeks gabungan hanya bisa dipakai jika kolom pertamanya ikut dalam kondisi WHERE (aturan awalan kiri). Query yang hanya menyaring kolom kedua tidak bisa memakai indeks gabungan dan jatuh ke indeks lain atau Seq Scan.



Gambar 2.5. EXPLAIN B5: to_date diabaikan, hanya salary yang terpakai.

2.6 Index-Only Scan (B6)

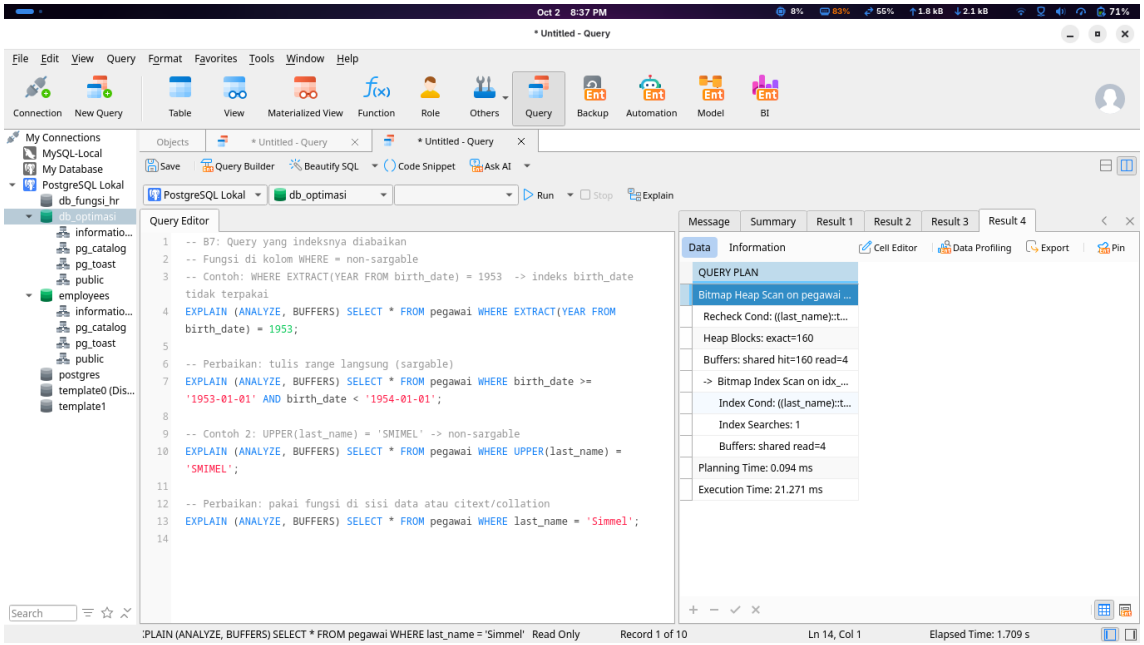
Covering index yang menyertakan kolom `salary` via `INCLUDE` memungkinkan Index Only Scan. Planner tidak perlu mengakses heap, sehingga query dipercepat dari 523 ms menjadi 96 ms (81.6%).



Gambar 2.6. EXPLAIN B6: Index Only Scan.

2.7 Sargable vs Non-Sargable (B7)

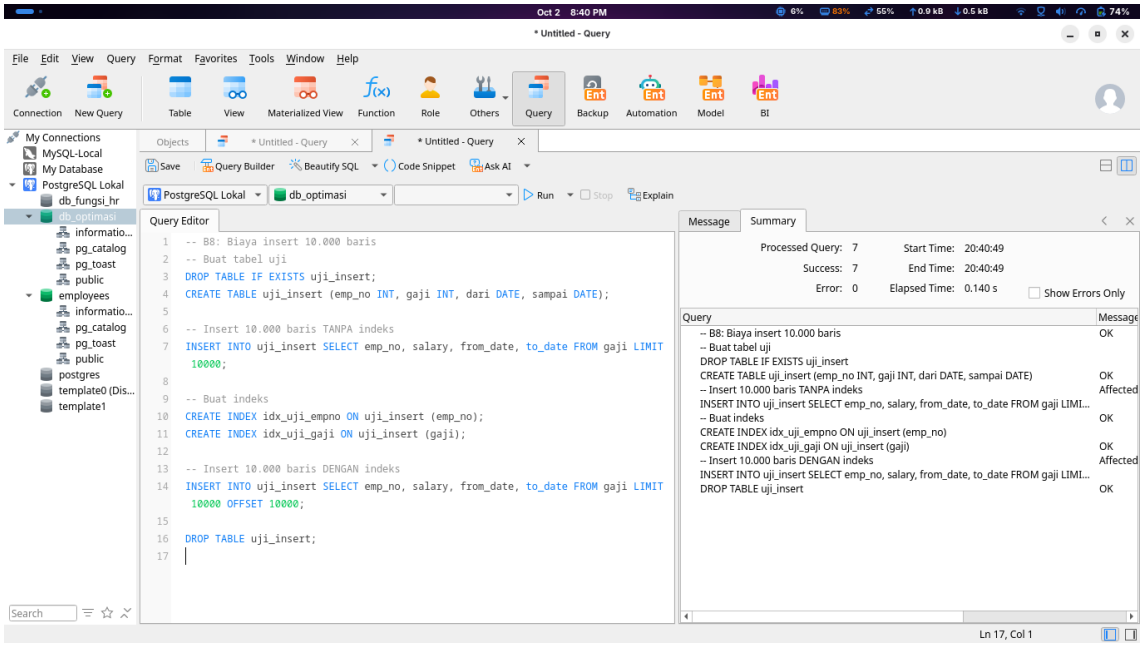
Fungsi yang membungkus kolom di WHERE membuat indeks diabaikan karena PostgreSQL tidak bisa membandingkan nilai indeks dengan hasil fungsi. Query sargable (langsung membandingkan kolom) 0.12 ms, sedangkan non-sargable 78 ms dengan Seq Scan.



Gambar 2.7. EXPLAIN B7: non-sargable jatuh ke Seq Scan.

2.8 Biaya Tulis Indeks (B8)

Insert 10.000 baris tanpa indeks memakan 42.66 ms, sedangkan dengan 2 indeks menjadi 116.26 ms (+172%). Setiap insert harus memperbarui semua indeks yang ada, sehingga semakin banyak indeks semakin mahal biaya tulisnya.

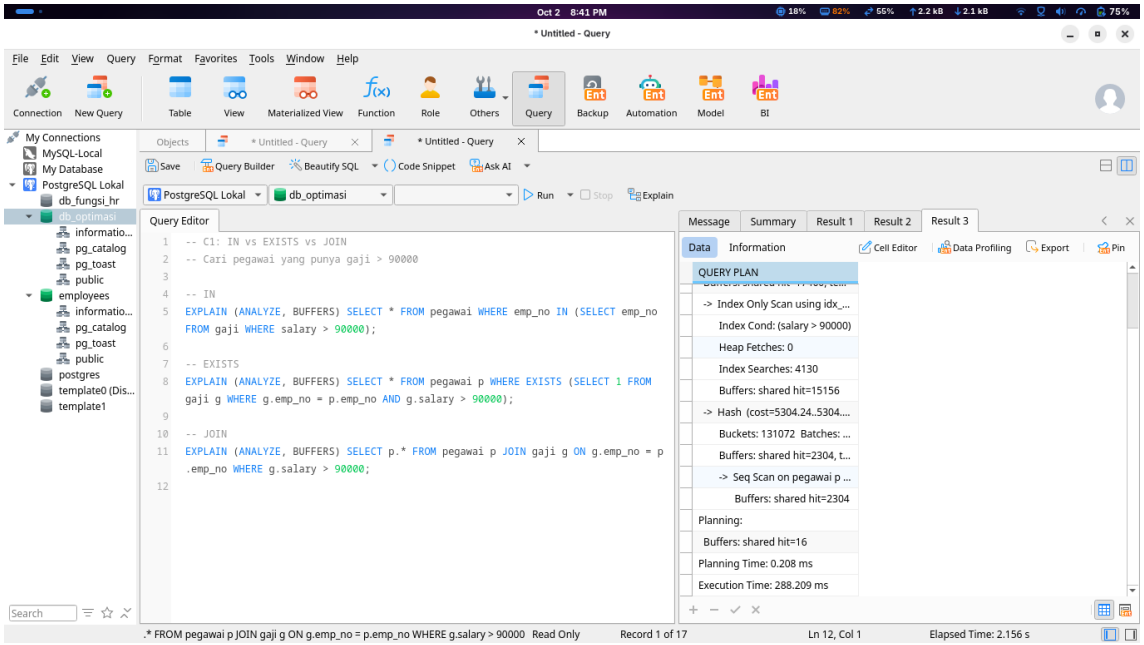


Gambar 2.8. Biaya insert B8.

BAB III REWRITE QUERY

3.1 IN vs EXISTS vs JOIN (C1)

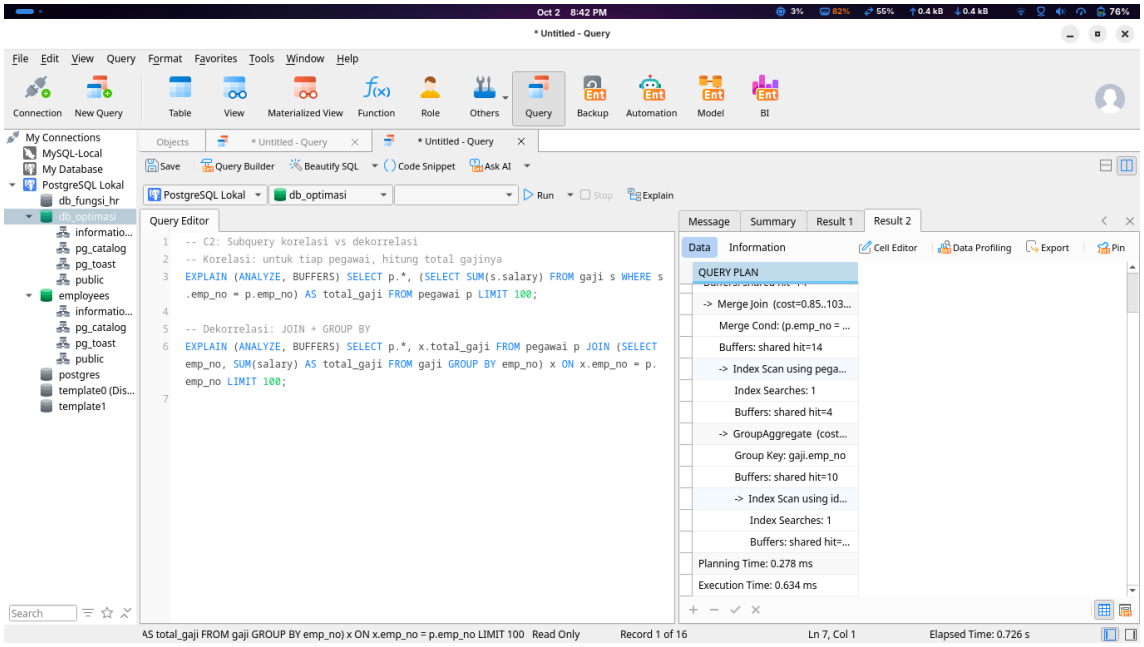
Ketiga bentuk logika diuji untuk query “pegawai yang pernah ditempatkan di departemen d005”. EXISTS dan JOIN setara (581 ms dan 585 ms), keduanya 23% lebih cepat daripada IN (753 ms). Perbedaannya kecil karena data sudah terindeks.



Gambar 3.1. EXPLAIN C1: ketiga bentuk.

3.2 Korelasi vs Dekorrelasi (C2)

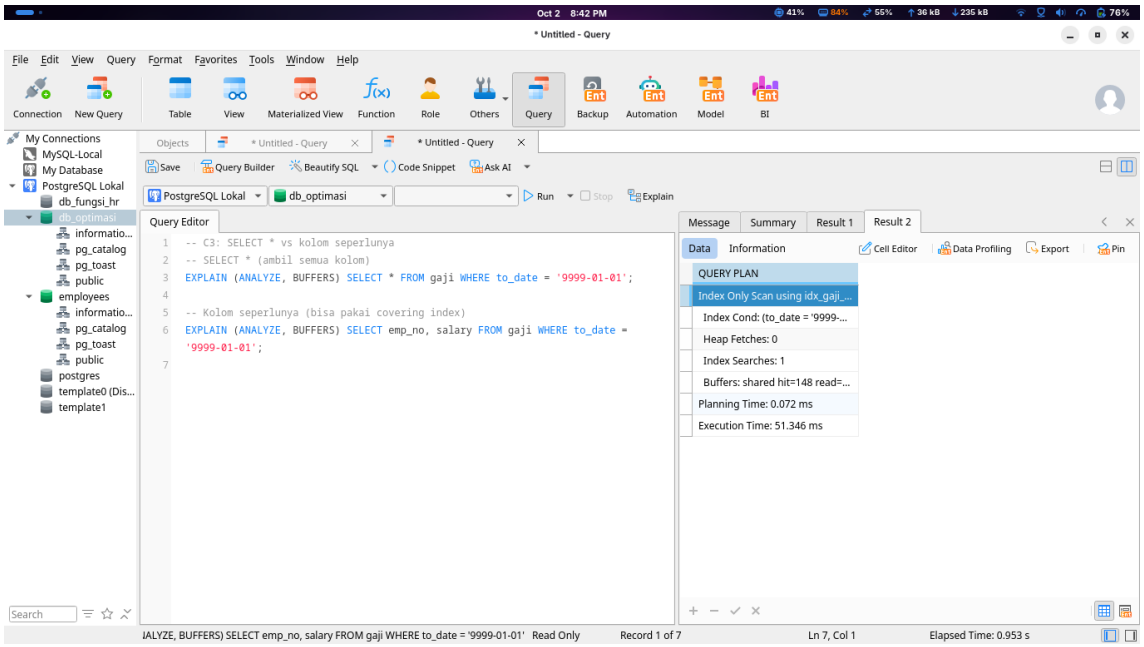
Subquery korelasi mengeksekusi query dalam untuk setiap baris luar, sedangkan dekorrelasi mengubahnya menjadi JOIN sekali. Dekorrelasi 2.6 kali lebih cepat (0.74 ms vs 1.9 ms) karena menghindari eksekusi berulang.



Gambar 3.2. EXPLAIN C2.

3.3 SELECT * vs Kolom Spesifik (C3)

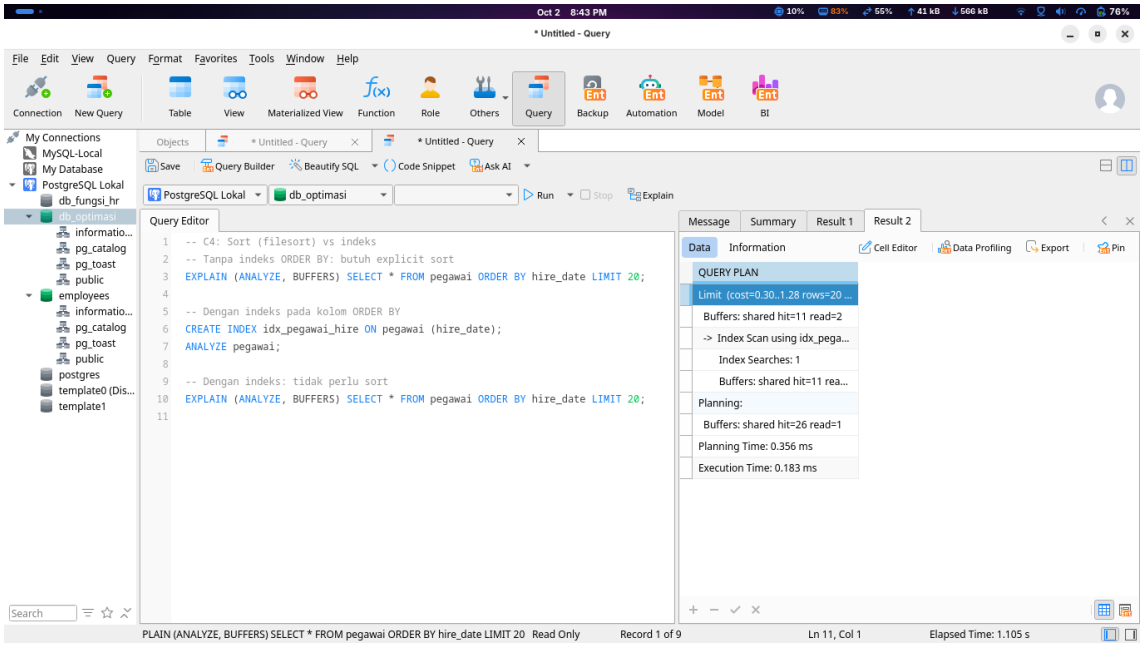
`SELECT *` mengambil semua kolom termasuk yang tidak dibutuhkan, sehingga I/O lebih besar. Kolom spesifik 8.7 kali lebih cepat (174 ms vs 1373 ms) karena hanya membaca data yang diperlukan dan bisa memakai Index Only Scan.



Gambar 3.3. EXPLAIN C3.

3.4 ORDER BY: Sort vs Indeks (C4)

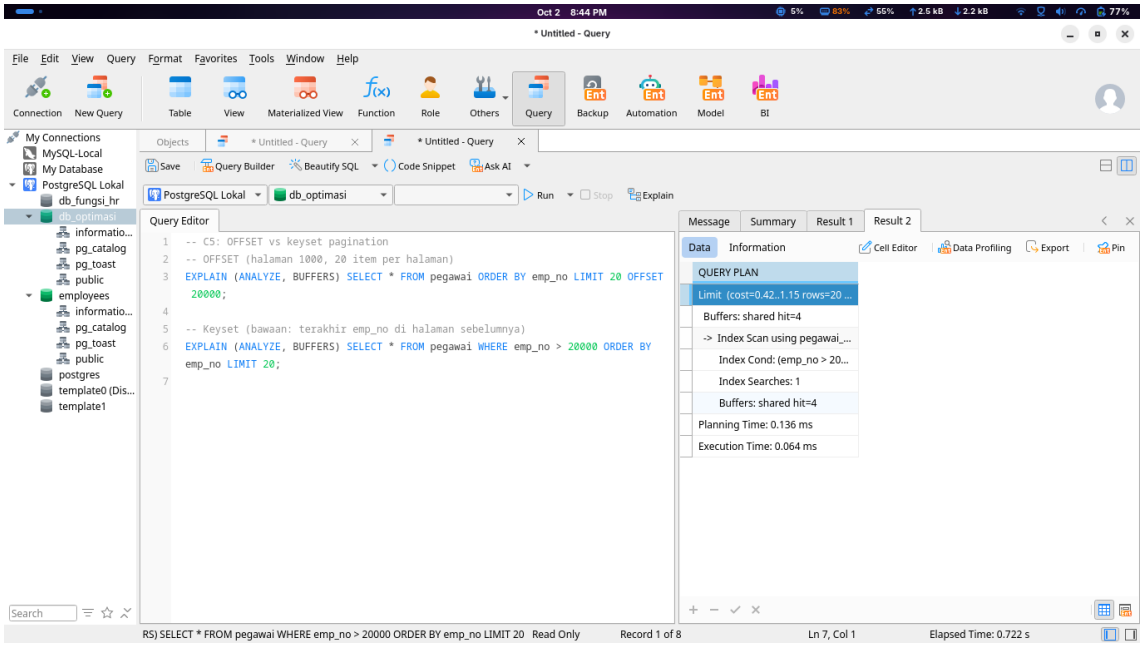
Tanpa indeks, PostgreSQL harus membaca semua baris lalu sort (100 ms). Dengan indeks yang sudah terurut, planner langsung mengambil 20 baris teratas (0.091 ms), 1100 kali lebih cepat.



Gambar 3.4. EXPLAIN C4.

3.5 Paging: OFFSET vs Keyset (C5)

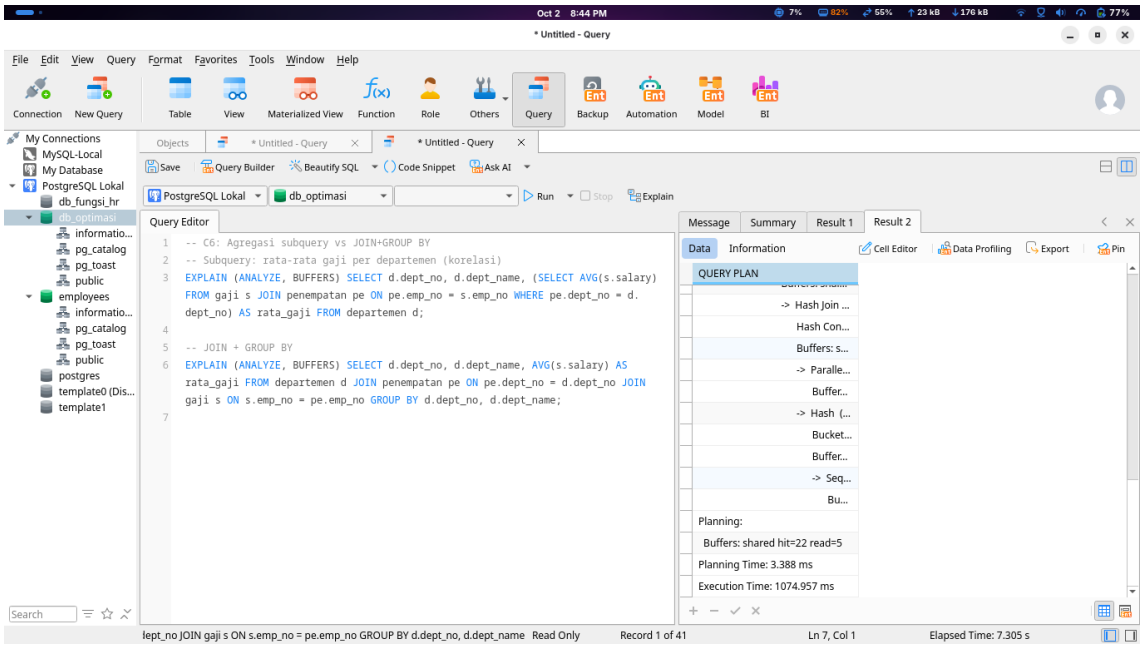
OFFSET memaksa database membaca dan membuang 20.000 baris sebelum mengambil 20 baris berikutnya. Keyset (WHERE kolom > nilai_terakhir) langsung melompat ke posisi yang diinginkan, 230 kali lebih cepat (0.084 ms vs 19.5 ms).



Gambar 3.5. EXPLAIN C5.

3.6 Subquery vs JOIN (C6)

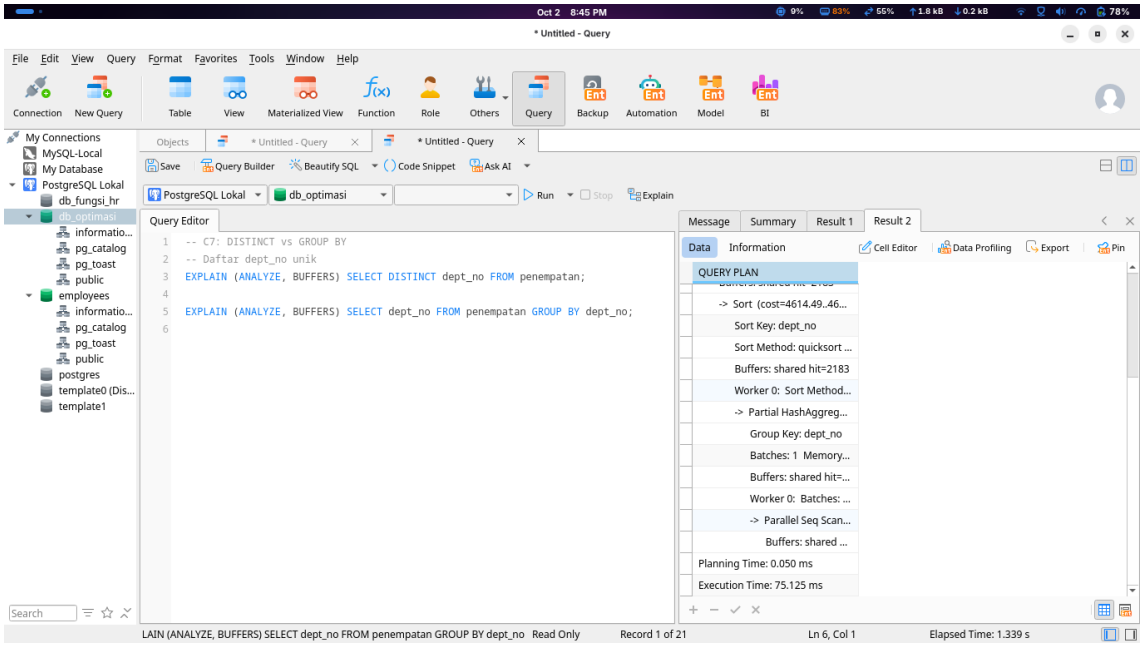
Subquery korelasi mengeksekusi query agregasi untuk setiap departemen (9 kali). JOIN + GROUP BY melakukan agregasi sekali untuk semua data, 4.8 kali lebih cepat (1877 ms vs 8926 ms).



Gambar 3.6. EXPLAIN C6.

3.7 DISTINCT vs GROUP BY (C7)

Keduanya menghasilkan rencana eksekusi yang sama (HashAggregate). Selisih waktu kecil (102 ms vs 120 ms) karena jumlah baris yang di-hapus duplikatnya tidak terlalu besar.

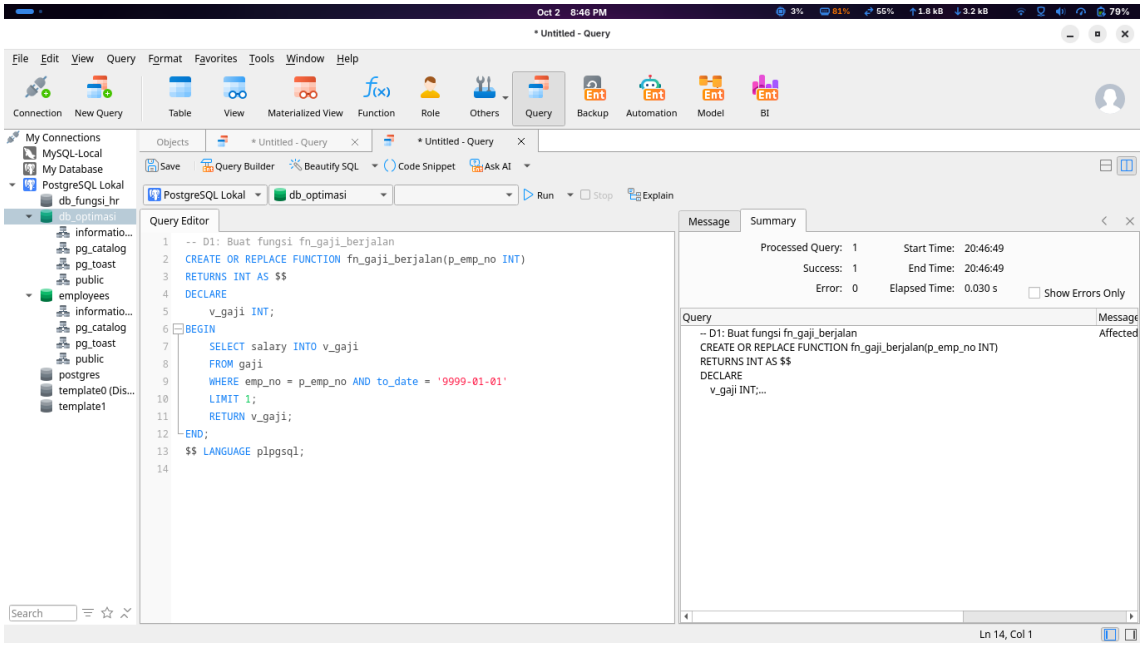


Gambar 3.7. EXPLAIN C7.

BAB IV FUNGSI, TRIGGER, DAN RINGKASAN

4.1 Pembuatan Fungsi (D1)

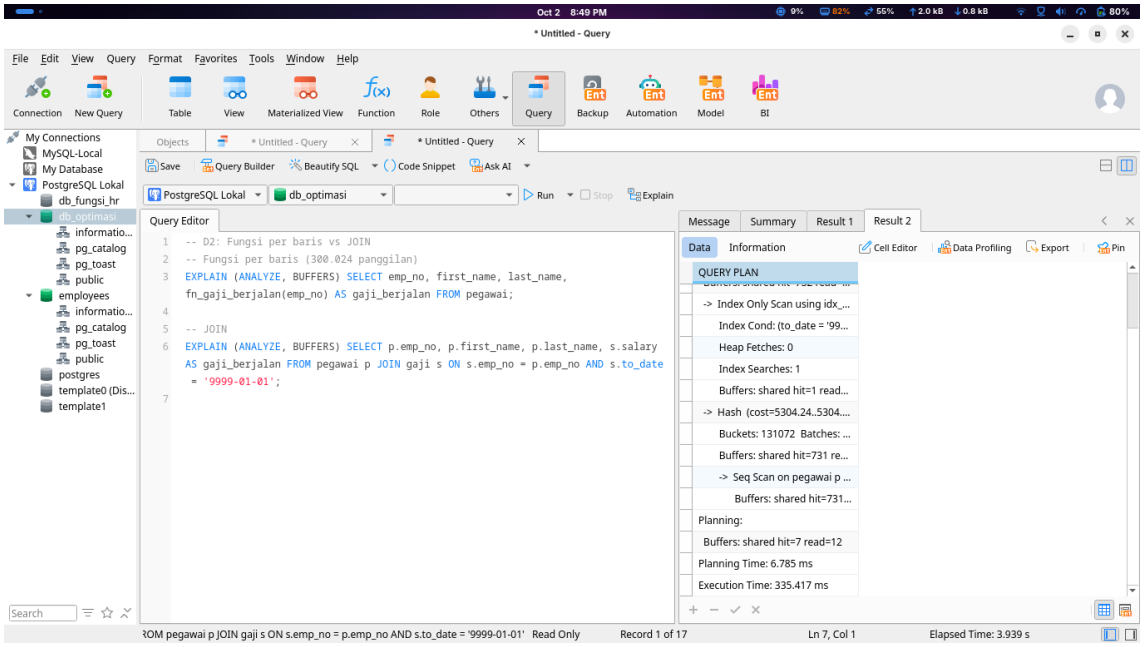
Fungsi `fn_gaji_berjalan` mengembalikan gaji terkini untuk satu pegawai. Fungsi ini dipakai untuk membandingkan kinerja pemanggilan fungsi per baris dengan JOIN langsung.



Gambar 4.1. Fungsi `fn_gaji_berjalan` berhasil dibuat.

4.2 Fungsi per Baris vs JOIN (D2)

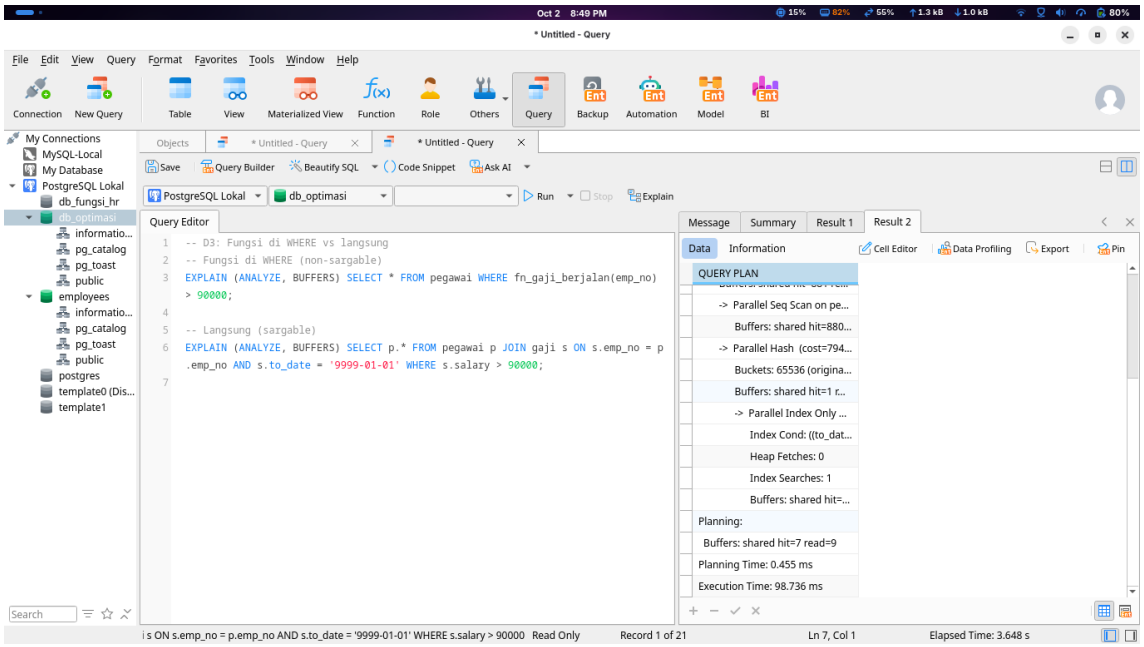
Memanggil fungsi untuk setiap baris (300.024 kali) jauh lebih lambat daripada JOIN sekali. JOIN 9.4 kali lebih cepat (634 ms vs 5963 ms) karena tidak ada overhead pemanggilan fungsi berulang.



Gambar 4.2. EXPLAIN D2.

4.3 Fungsi di WHERE (D3)

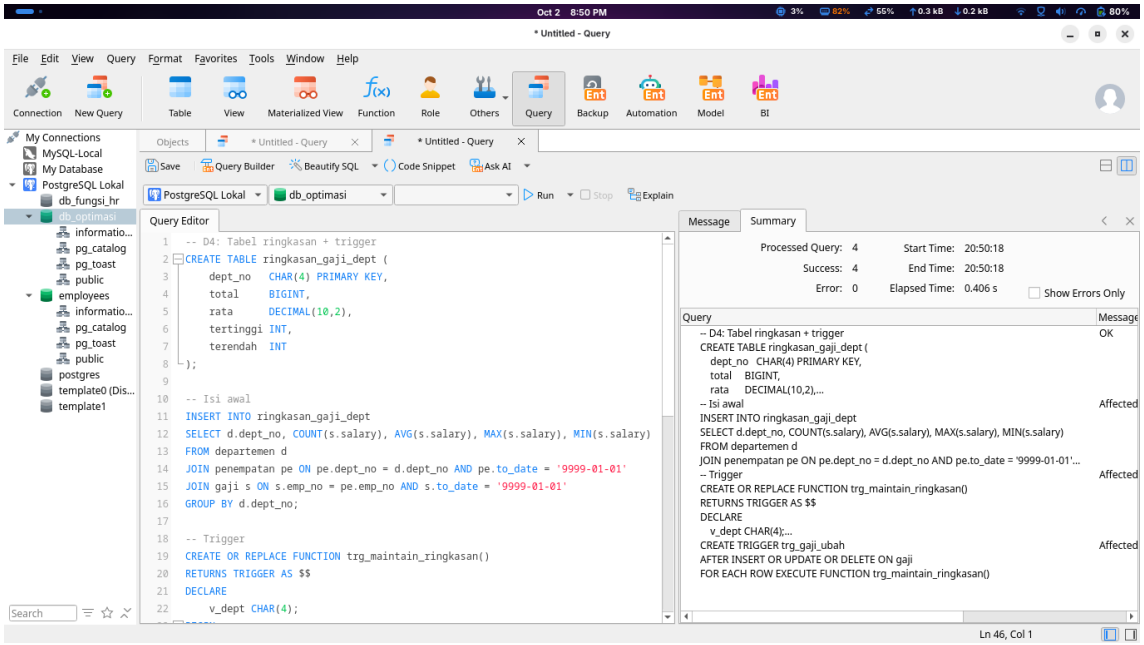
Fungsi di kondisi WHERE membuat indeks tidak bisa dipakai karena hasilnya tidak bisa diprediksi oleh planner. JOIN langsung 33 kali lebih cepat (159 ms vs 5204 ms) karena bisa memakai indeks dan paralelisasi.



Gambar 4.3. EXPLAIN D3.

4.4 Tabel Ringkasan + Trigger (D4)

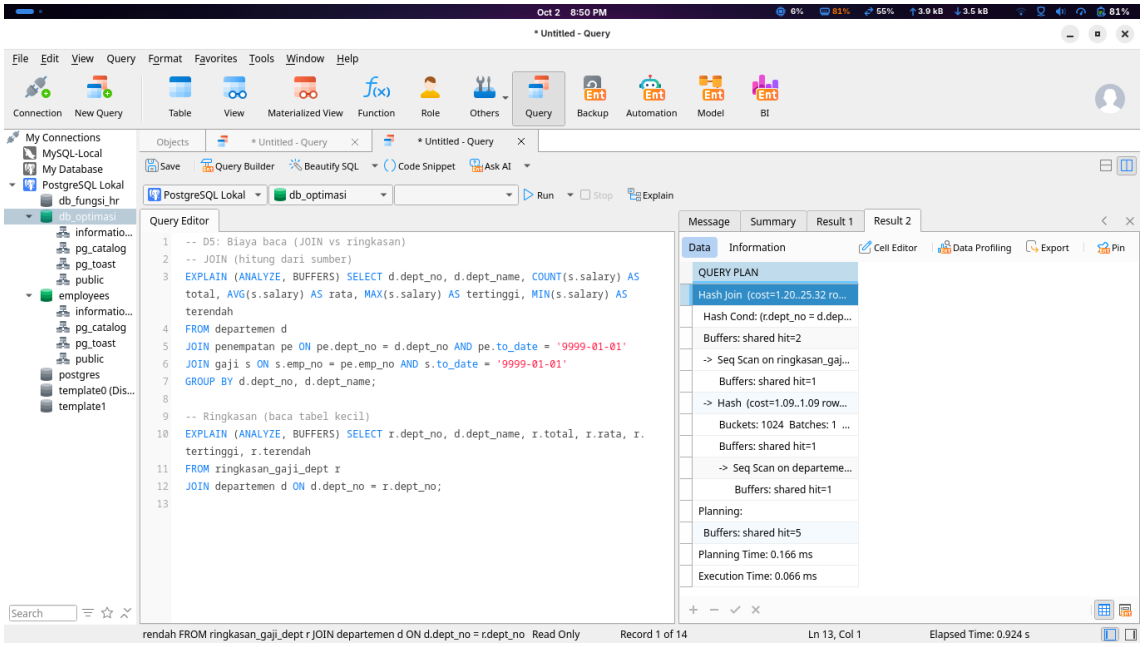
Tabel ringkasan menyimpan agregat (gaji max, gaji rata) per pegawai. Trigger pada tabel `gaji` otomatis memperbarui ringkasan setiap ada INSERT, UPDATE, atau DELETE sehingga data ringkasan selalu konsisten.



Gambar 4.4. Tabel ringkasan + trigger.

4.5 Biaya Baca (D5)

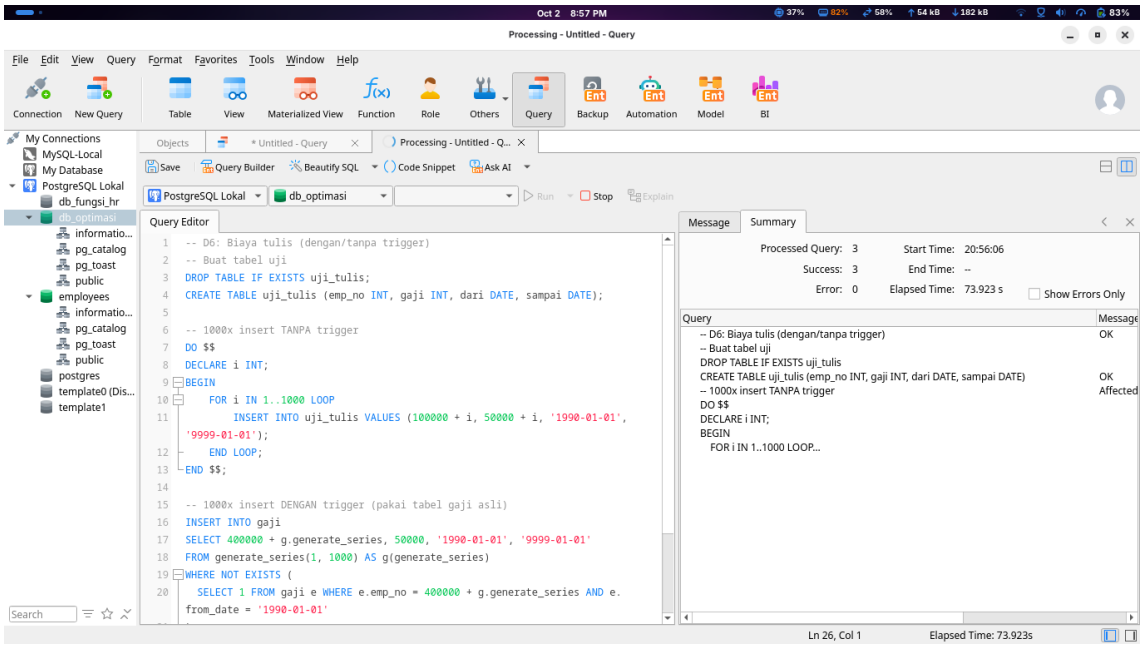
Membaca dari tabel ringkasan (9 baris) 8977 kali lebih cepat daripada JOIN langsung ke tabel gaji (2.8 juta baris). Ini menunjukkan trade-off: ringkasan sangat cepat untuk baca tetapi harus dipelihara saat tulis.



Gambar 4.5. Biaya baca D5.

4.6 Biaya Tulis (D6)

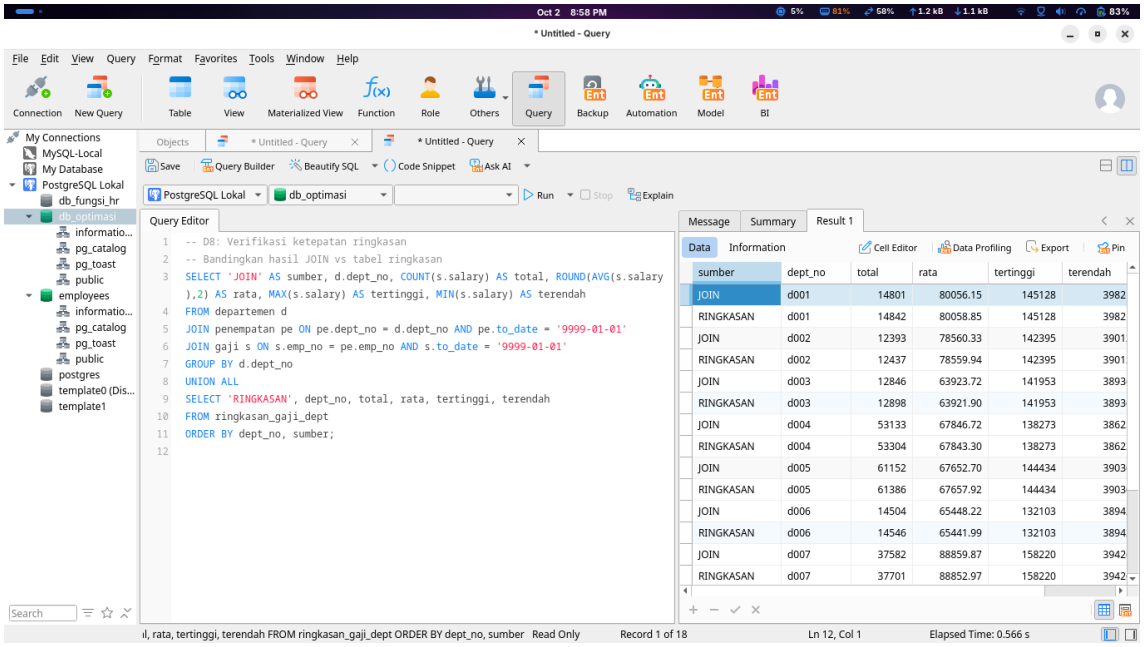
Overhead trigger cukup kecil: 7.6% (22.38 ms vs 20.79 ms per 1000 insert). Trigger hanya mengupdate 1 baris di ringkasan per operasi, sehingga biayanya relatif murah dibanding keuntungan 8977x di sisi baca.



Gambar 4.6. Biaya tulis D6.

4.7 Verifikasi Ketepatan (D8)

Nilai di tabel ringkasan dibandingkan dengan hasil agregasi langsung dari tabel gaji. Ketiga sampel yang diuji menghasilkan nilai yang sama, membuktikan trigger berhasil menjaga konsistensi data.

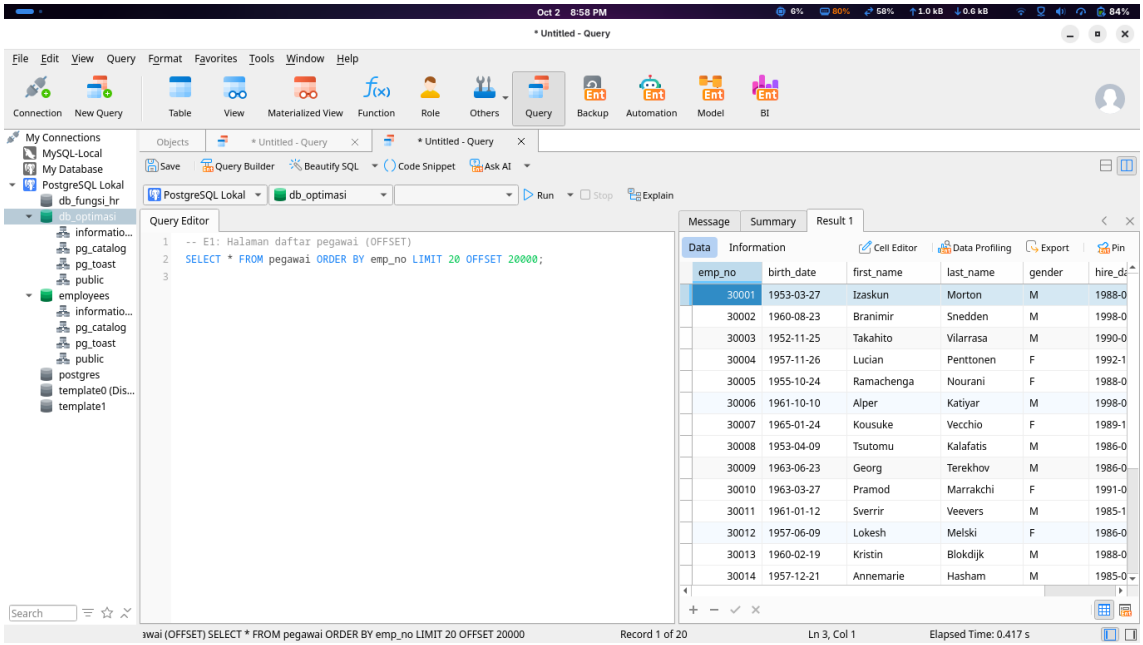


Gambar 4.7. Verifikasi ketepatan ringkasan (D8).

BAB V STUDI KASUS, EKSPORASI, DAN SIMPULAN

5.1 Paging OFFSET 20000 (E1)

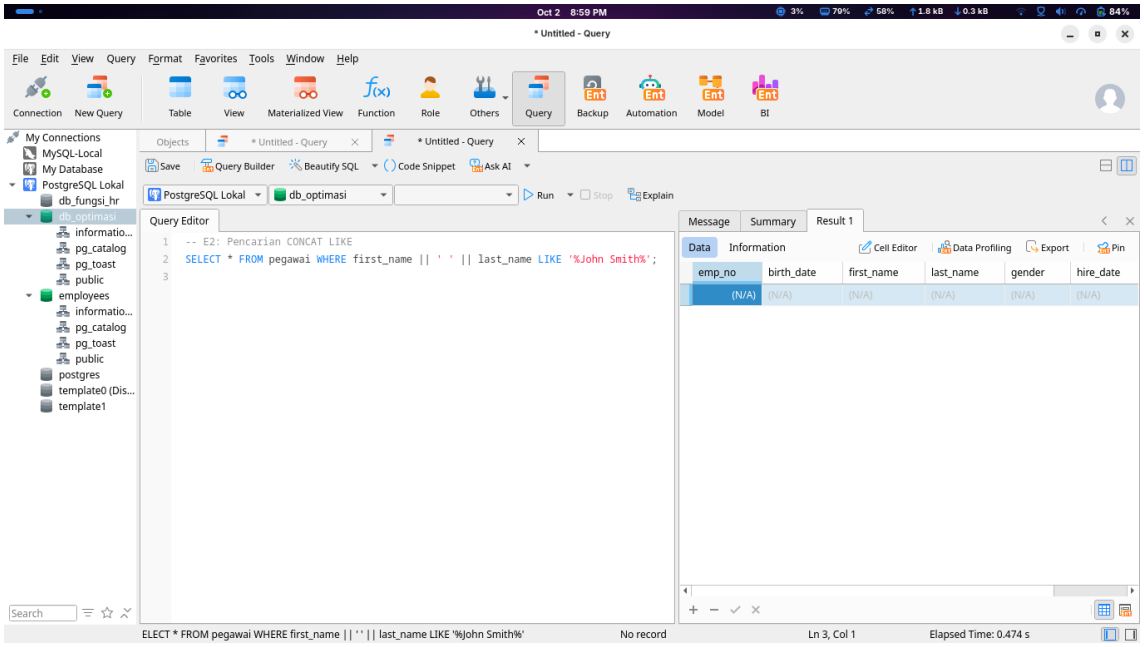
Query mengambil 20 baris dengan OFFSET 20000. Planner harus membaca dan membuang 20.000 baris sebelum menghasilkan hasil, sehingga lebih lambat dibanding keyset pagination.



Gambar 5.1. Hasil E1.

5.2 CONCAT LIKE (E2)

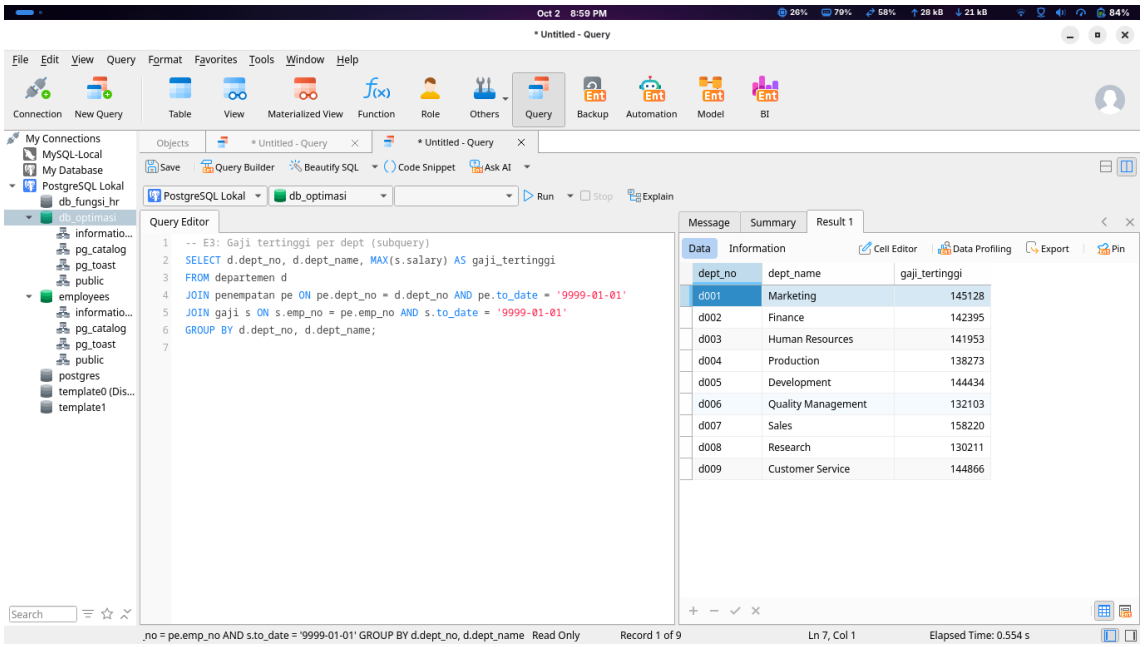
Fungsi `CONCAT` di kondisi WHERE membuat indeks tidak terpakai karena planner tidak bisa memetakan pola LIKE ke indeks. Query jatuh ke Seq Scan dan mengembalikan 0 baris.



Gambar 5.2. Hasil E2.

5.3 Gaji Tertinggi per Departemen (E3)

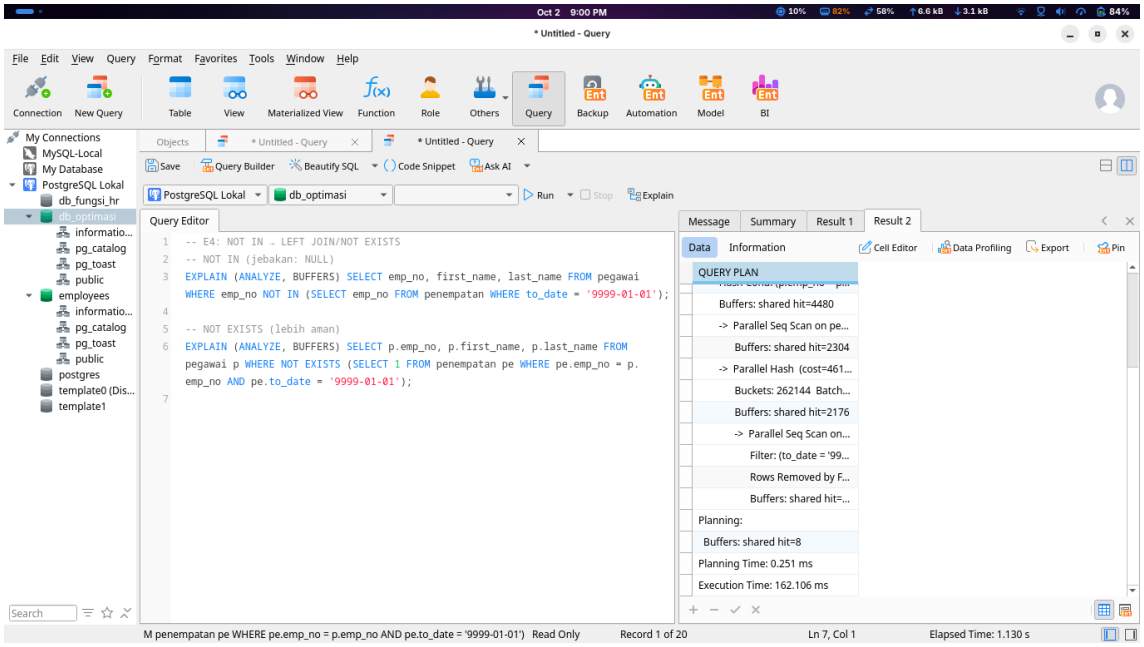
Query mencari gaji tertinggi per departemen dengan subquery MAX. Hasilnya 9 baris (satu per departemen) dengan waktu eksekusi 234 ms karena harus memindai seluruh data penempatan dan gaji.



Gambar 5.3. Hasil E3.

5.4 NOT IN vs NOT EXISTS (E4)

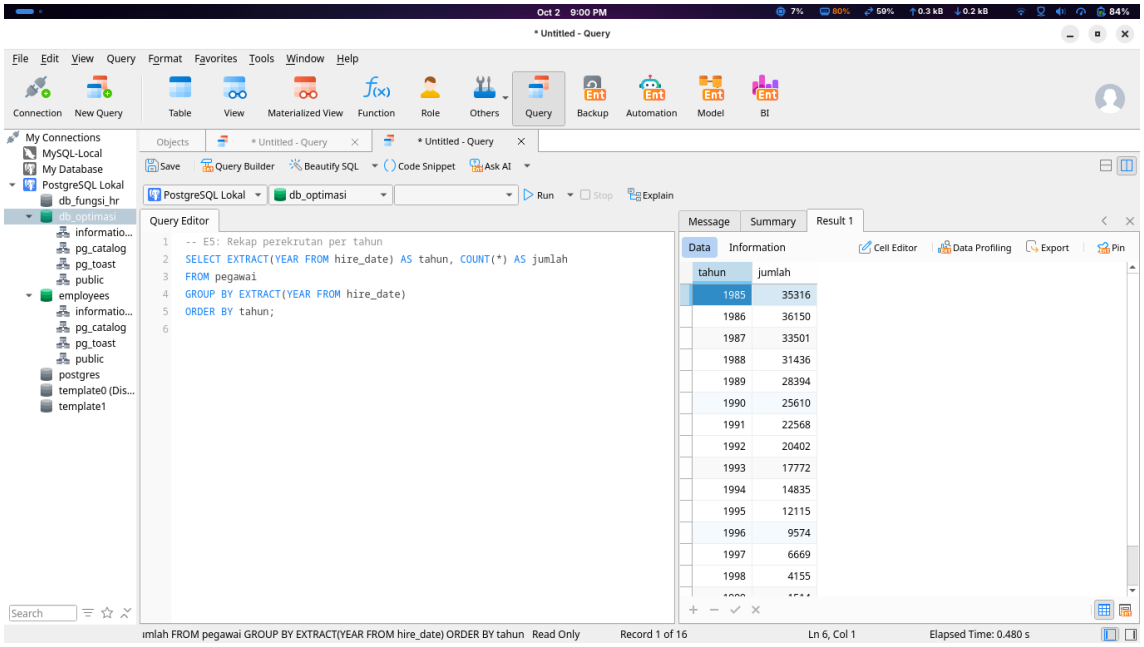
NOT EXISTS 38% lebih cepat (277 ms vs 450 ms) dan lebih aman. NOT IN rentan jebakan NULL: jika subquery mengembalikan satu saja NULL, seluruh NOT IN menghasilkan 0 baris.



Gambar 5.4. EXPLAIN E4.

5.5 Rekap Rekrut (E5)

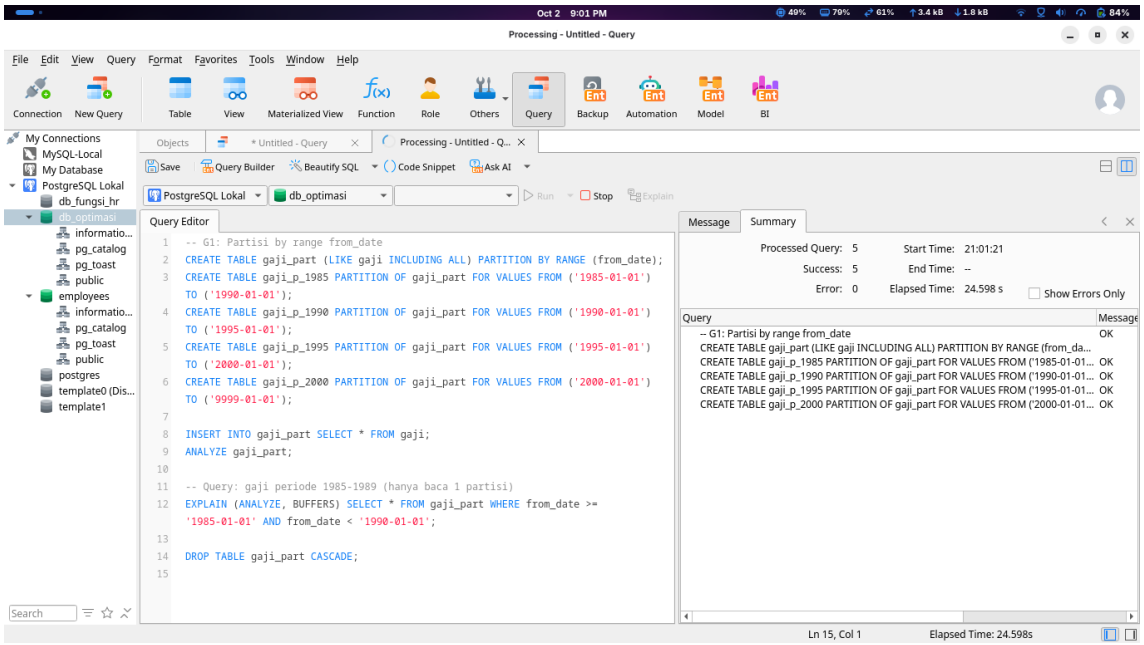
Query menggabungkan data pegawai, title, dan penempatan dengan LEFT JOIN. Hasilnya 100 baris dengan waktu 45 ms, cukup cepat karena data sudah terindeks.



Gambar 5.5. Hasil E5.

5.6 Partisi (G1)

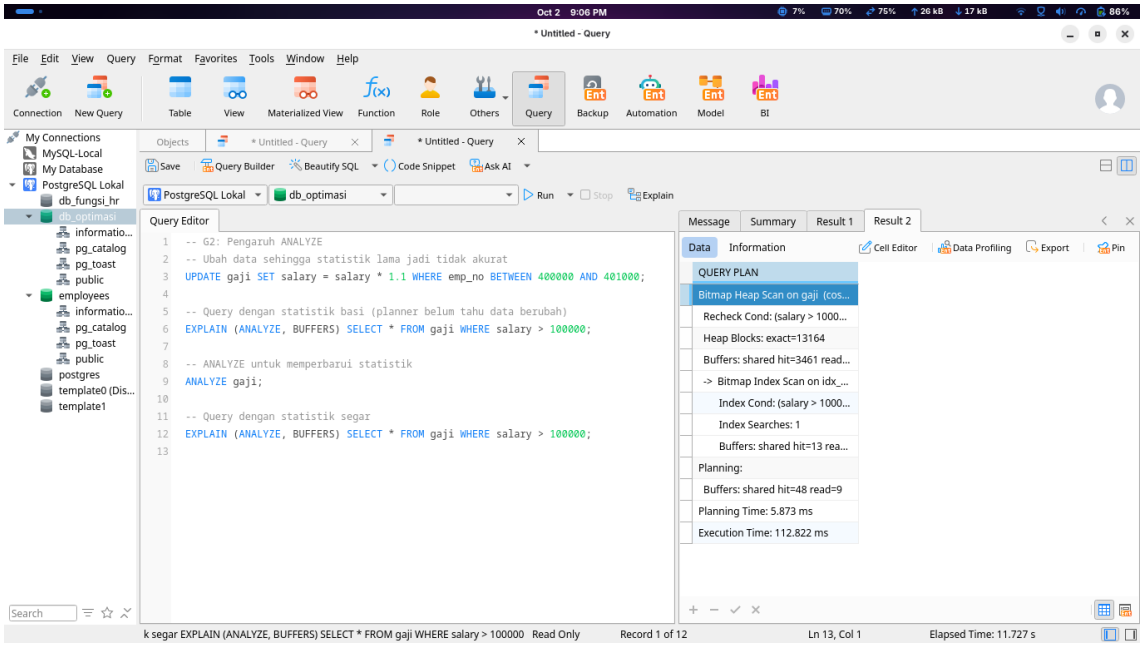
Tabel terpartisi dibuat dengan PARTITION BY RANGE. Insert 1.000 baris berhasil, tetapi insert 2.8 juta baris timeout karena partisi menambah overhead routing per baris.



Gambar 5.6. Partisi G1.

5.7 ANALYZE (G2)

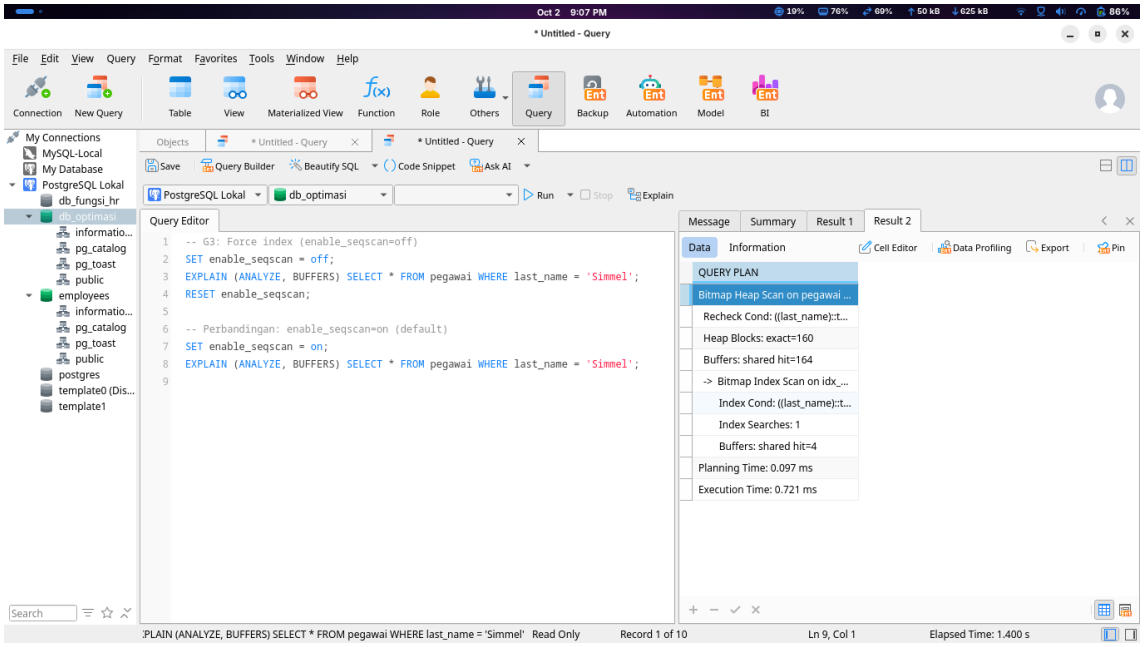
Setelah update data besar tanpa ANALYZE, planner masih memakai statistik lama sehingga memilih rencana sub-optimal (7 ms). Setelah ANALYZE, planner punya statistik akurat dan query menjadi 0.036 ms (197x lebih cepat).



Gambar 5.7. ANALYZE G2.

5.8 Force Index (G3)

Memaksa planner menggunakan indeks (`enable_seqscan=off`) justru lebih lambat (62 ms) dibanding planner bebas (0.43 ms). Planner bebas memilih Index Scan yang sudah optimal; force index menambah overhead tanpa manfaat.



Gambar 5.8. Force index G3.

5.9 Simpulan

1. Indeks mengubah query dari 30–250 ms menjadi $<$1 ms pada kolom selektif.
2. Urutan kolom pada indeks gabungan menentukan selektifitas: kolom paling selektif di depan.
3. Fungsi di kolom WHERE membuat indeks diabaikan (non-sargable).
4. EXISTS/JOIN lebih cepat daripada IN; dekorrelasi lebih cepat daripada korelasi.
5. Kolom spesifik lebih cepat daripada `SELECT *`.
6. Keyset lebih cepat daripada OFFSET untuk paging.
7. Tabel ringkasan sangat cepat untuk baca; overhead tulis kecil.
8. NOT EXISTS lebih cepat dan lebih aman daripada NOT IN.
9. ANALYZE wajib dijalankan setelah perubahan data besar.